

O'REILLY®

Practical MLOps

Operationalizing Machine Learning Models



Noah Gift & Alfredo Deza

Practical MLOps

Operationalizing Machine Learning Models

Noah Gift and Alfredo Deza

Practical MLOps

by Noah Gift and Alfredo Deza

Copyright © 2021 Noah Gift and Alfredo Deza. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Rebecca Novack

Development Editor: Melissa Potter

Production Editor: Daniel Elfanbaum

Copyeditor: Kim Cofer

Proofreader: Piper Editorial Consulting, LLC

Indexer: WordCo Indexing Services, Inc.

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

September 2021: First Edition

Revision History for the First Edition

- 2021-09-14: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098103019> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Practical MLOps*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors, and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-10301-9

[LSI]

Preface

Why We Wrote This Book

We've both spent most of our careers automating things. When we first met and Alfredo didn't know Python, Noah suggested automating one task per week. Automation is a core pillar for MLOps, DevOps, and this book throughout. You should take all the examples and opinions in this book in the context of future automation.

If Noah could summarize how he spent 2000–2020, it was automating just about anything he could, from film pipelines to software installation to machine learning pipelines. As an engineering manager and CTO at startups in the Bay Area, he built many data science teams from scratch. As a result, he saw many of the core problems in getting machine learning to production in the early stages of the AI/ML revolution.

Noah has been an adjunct professor at Duke, Northwestern, and UC Davis in the last several years, teaching topics that primarily focus on cloud computing, data science, and machine learning engineering. This teaching and work experience gives him a unique perspective about the issues involved in the real-world deployment of machine learning solutions.

Alfredo has a heavy ops background from his Systems Administrator days, with a similar passion for automation. It is not possible to build resilient infrastructure without push-button automation. There is nothing more gratifying when disaster situations happen than rerunning a script or a pipeline to re-create what crashed.

When COVID-19 hit, it accelerated a question we both had, which was, “why aren't we putting more models into production?” Noah touched on some of these issues in an [article he wrote for Forbes](#). The

summarized premise of the article is that something is wrong with data science because organizations are not seeing returns on their investments.

Later at O'Reilly's "Foo Camp", Noah led a session on "Why can we not be 10X faster at ML in production?" where we had a great discussion with many people, including Tim O'Reilly, Mike Loukides, Roger Magoulas, and others. The result of that discussion was: "Yes, we can go 10X faster." So thanks to Tim and Mike for stirring such a fascinating discussion and getting this book on its way.

Machine learning feels a lot like many other technologies that have appeared in the past several decades. At first, it takes years to get results. Steve Jobs talked about how NeXT wanted to make it 10X faster to build software (and he did). You can watch the interview on [YouTube](#). What are some of the problems with machine learning currently?

- Focus on the "code" and technical details versus the business problem
- Lack of automation
- HiPPO (Highest Paid Person's Opinions)
- Not cloud native
- Lack of urgency to solve solvable problems

Quoting one of the things Noah brought up in the discussion: "I'm anti-elitism across the board. Programming is a human right. The idea that there is some priesthood that is only allowed to do it is just wrong." Similar to machine learning, it is too crucial for technology to lie only in the hands of a select group of people. With MLOps and AutoML, these technologies can go into the public's hands. We can do better with machine learning and artificial intelligence by democratizing this technology. "Real" AI/ML practitioners ship models to production, and in the "real" future, people such as doctors, lawyers, mechanics, and teachers will use AI/ML to help them do their jobs.

How This Book Is Organized

We designed this book so that you can consume each chapter as a standalone section designed to give you immediate help. At the end of each chapter are discussion questions that are intended to spur critical thinking and technical exercises to improve your understanding of the material.

These discussion questions and exercises are also well suited for use in the classroom in a Data Science, Computer Science, or MBA program and for the motivated self-learner. The final chapter contains several case studies helpful in building a work portfolio as an expert in MLOps.

The book is divided into 12 chapters, which we'll break down a little more in the following section. At the end of the book, there is an appendix with a collection of valuable resources for implementing MLOps.

Chapters

The first few chapters cover the theory and practice of both DevOps and MLOps. One of the items covered is how to set up continuous integration and continuous delivery. Another critical topic is Kaizen, i.e., the idea of continuous improvement in everything.

There are three chapters on cloud computing that cover AWS, Azure, and GCP. Alfredo, a developer advocate for Microsoft, is an ideal source of knowledge for MLOps on the Azure platform. Likewise, Noah has spent years getting students trained on cloud computing and working with the education arms of Google, AWS, and Azure. These chapters are an excellent way to get familiar with cloud-based MLOps.

Other chapters cover critical technical areas of MLOps, including AutoML, containers, edge computing, and model portability. These topics encompass many cutting-edge emerging technologies with active traction.

Finally, in the last chapter, Noah covers a real-world case study of his time at a social media startup and the challenges they faced doing MLOps.

Appendixes

The appendixes are a collection of essays, ideas, and valuable items that cropped up in years between finishing *Python for DevOps* (O'Reilly) and this book. The primary way to use them is to help you make decisions about the future.

Exercise Questions

In this book's exercises, a helpful heuristic considers how you can leverage them into a portfolio using GitHub and a YouTube walkthrough of what you did. In keeping with the expression "a picture is worth a thousand words," a YouTube link to a walkthrough of a reproducible GitHub project on a resume may be worth 10,000 words and puts the resume in a new category of qualification for a job.

As you go through the book and exercises, consider the following critical thinking framework.

Discussion Questions

According to Jonathan Haber in *Critical Thinking* (MIT Press Essential Knowledge series) and the nonprofit **Foundation for Critical Thinking**, discussion questions are essential critical thinking components. The world is in dire need of critical thinking due to the proliferation of misinformation and shallow content in social media. A mastery of the following skills sets an individual apart from the pack:

Intellectual humility

Recognition of the limits of your knowledge.

Intellectual courage

The ability to argue for your beliefs even in the face of social pressure.

Intellectual empathy

The ability to put yourself in the minds of others to understand their position.

Intellectual autonomy

The ability to think for yourself independently of others.

Intellectual integrity

The ability to think and argue with the same intellectual standards you expect others to apply to you.

Intellectual perseverance

The ability to provide evidence that supports your position.

Confidence in reason

The belief that there are indisputable facts and that reason is the best solution to gain knowledge.

Fairmindedness

The ability to put in the good-faith effort to treat all viewpoints fairly.

Using these criteria, evaluate the discussion questions in each chapter.

Origin of Chapter Quotes

By Noah

I graduated college in late 1998 and spent a year training to play professional basketball in the minor leagues in the United States or Europe while working as a personal trainer. My backup plan was to get a job in IT. I applied to be a Systems Administrator at Caltech in Pasadena and got a Mac IT expert position on a fluke. I decided the risk/reward ratio of being a low-paid professional athlete wasn't worth it and accepted the job offer.

To say Caltech changed my life is an understatement. At lunch, I played ultimate frisbee and heard about the Python programming language, which I learned so I would "fit in" with my ultimate frisbee friends, who were staff or students at Caltech. Later, I worked directly for Caltech's administration and was the personal Mac expert at Caltech for Dr. David Baltimore, who got the Nobel Prize in his 30s. I interacted with many famous people in many unexpected ways, which boosted my self-confidence and grew my network.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

I also had many Forrest Gump–style random encounters with people who would later do incredible things in AI/ML. Once, I had dinner with Dr. Fei-Fei Li, head of AI at Stanford, and her boyfriend; I remember being impressed that her boyfriend spent the summer writing a video game with his dad. I was highly impressed and thought, “Who does that kind of thing?” Later, I set up a mail server under the famous physicist Dr. David Goodstein’s desk because he kept getting grief from IT about hitting his mailbox storage limits. These experiences are where I acquired a taste for building “shadow infrastructure.” Because I worked directly for the administration, I got to flaunt the rules if there was a good reason for it.

One of the people I randomly met was Dr. Joseph Bogen, a neurosurgeon and visiting professor at Caltech. Of all the people I met at Caltech, he had the most profound impact on my life. One day I responded to a help desk call to come to his house to fix his computer, and later this turned into a weekly dinner at his home with him and his wife, Glenda. From around 2000 until the day he died, he was a friend and mentor.

At the time, I was very interested in artificial intelligence, and I remember a Caltech Computer Science professor telling me it was a dead field and I shouldn’t focus on it. Despite that advice, I came up with a plan to be fluent in many software programming languages by 40 years old and writing artificial intelligence programs by then. Lo and behold, my plan worked out.

I can clearly say I wouldn’t be doing what I am doing today if I didn’t meet Joe Bogen. He blew my mind when he told me he did the first hemispherectomy, removing half of a brain, to help a patient with severe epilepsy. We would talk for hours about the origins of consciousness, the use of neural networks in the 1970s to figure out who would be an Air Force pilot, and whether your brain contained “two of you,” one in each hemisphere. Above all, what Bogen gave me was a sense of confidence in my intellect. I had severe doubts up until that point about what I could do, but our conversations were like a master’s degree in higher-level thinking. As a professor myself, I think about how big of an impact he had on my life, and I hope to pay it forward to other students I interact with, both as a formal teacher or someone they meet. You can read these quotes yourself from an archive of [Dr. Bogen’s Caltech home page](#) and his [biography](#).

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

TIP

This element signifies a tip or suggestion.

NOTE

This element signifies a general note.

WARNING

This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://github.com/paiml/practical-mlops-book>.

If you have a technical question or a problem using the code examples, please send email to bookquestions@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Practical MLOps* by Noah Gift and Alfredo Deza (O'Reilly). Copyright 2021 Noah Gift and Alfredo Deza, 978-1-098-10301-9."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning

NOTE

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <http://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-998-9938 (in the United States or Canada)

707-829-0515 (international or local)

707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/practical-mlops>.

Email bookquestions@oreilly.com to comment or ask technical questions about this book.

For news and information about our books and courses, visit <http://oreilly.com>.

Find us on Facebook: <http://facebook.com/oreilly>.

Follow us on Twitter: <http://twitter.com/oreillymedia>.

Watch us on YouTube: <http://www.youtube.com/oreillymedia>.

Acknowledgments

From Noah

As mentioned earlier, without Mike Loukides inviting me to Foo Camp and having a great discussion with Tim O'Reilly and me, this book wouldn't be here. Next, I would like to acknowledge Alfredo, my coauthor. I have had the pleasure of writing five books, two for O'Reilly and three self-published, in a little over two years with Alfredo, and this is primarily due to his ability to embrace work and get things done. An appetite for hard work is perhaps the best talent, and Alfredo has this skill in abundance.

Our editor, Melissa Potter, did tremendous work getting things into shape, and the book before she edited and afterward are almost two different books. I feel lucky to have worked with such a talented editor.

Our technical editors, including Steve Depp, Nivas Durairaj, and Shubham Saboo, played a crucial role in giving us great feedback about where to zig and when to zag. Many enhancements are particularly due to Steve's thorough feedback. Also, I wanted to thank Julien Simon and Piero Molino for enhancing our book with real-world thoughts on MLOps.

I want to thank my family, Liam, Leah, and Theodore, for giving me the space to finish this book on a tight deadline in the middle of a pandemic. I am also looking forward to reading some of the books they write in the future. Another big group of thanks goes out to all the former students I taught at Northwestern, Duke, UC Davis, and other schools. Many of their questions and feedback made it into this book.

My final thanks go out to Dr. Joseph Bogen, an early pioneer in AI/ML and Neuroscience. If we didn't bump into each other at Caltech, there is zero chance I would be a professor or that this book would exist. His impact was that big on my life.

From Alfredo

I'm absolutely thankful for my family's support while writing this book: Claudia, Efrain, Ignacio, and Alana—your support and patience were essential to get to the finish line. Thanks again for all the opportunities to work with you, Noah; this was another incredible ride. I value your friendship and our professional relationship.

Thanks to Melissa Potter (without a doubt the best editor I've worked with) for her fantastic work. Our technical editors did great by finding problems and highlighting places that needed refinement, always a hard thing to do well.

Also extremely grateful for Lee Stott's help with Azure. The Azure content wouldn't be as good without it. And thanks to Francesca Lazzeri, Mike McCoy, and everyone else I contacted at Microsoft about the book. You were all very helpful.

Chapter 1. Introduction to MLOps

By Noah Gift

Since 1986, I have had a few more deaths, several from insufficient attention but mainly from deliberately pushing the limits in various directions—taking a chance in bonsai is a bit like taking a chance with love; the best outcome requires risky exposure to being hurt and no guarantee of success.

—Dr. Joseph Bogen

One of the powerful aspects of science fiction is its ability to imagine a future without constraints. One of the most influential science fiction shows of all time is the TV show *Star Trek*, which first aired in the mid-1960s, approximately 60 years ago. The cultural impact inspired designers of technology like the Palm Pilot and hand-held cellular phones. In addition, *Star Trek* influenced the cofounder of Apple computer, Steve Wozniak, to create Apple computers.

In this age of innovation in machine learning, there are many essential ideas from the original series relevant to the coming MLOps (or Machine Learning Operations) industrial revolution. For example, *Star Trek* hand-held tricorders can instantly classify objects using pretrained multiclass classification models. But, ultimately, in this futuristic science-fiction world, domain experts like the science officers, medical officers, or captain of the ship, don't spend months training machine learning models. Likewise, the crew of their science vessel, the *Enterprise*, are not called data scientists. Instead, they have jobs where they often use data science.

Many of the machine learning aspects of this *Star Trek* science fiction future are no longer science fiction in the 2020s. This chapter guides the reader into the foundational theory of how to make this possible. Let's get started.

Rise of the Machine Learning Engineer and MLOps

Machine learning (ML), with its widespread adoption globally, has created a need for a systematic and efficient approach toward building ML systems, leading to a sudden rise in demand for ML engineers. These ML engineers, in turn, are applying established DevOps best practices to the emerging machine learning technologies. The major cloud vendors all have certifications targeting these practitioners. I have experience working directly with AWS, Azure, and GCP as a subject matter expert on machine learning. In some cases, this includes helping create machine learning certifications themselves and official training material. In addition, I teach machine learning engineering and cloud computing at some of the top data science programs with Duke and Northwestern. Firsthand, I have seen the rise in machine learning engineering as many former students have become machine learning engineers.

Google has a **Professional Machine Learning Engineer certification**. It describes an ML engineer as someone who “designs, builds, and productionizes ML models to solve business challenges...” Azure has a **Microsoft Certified: Azure Data Scientist Associate**. It describes this type of practitioner as someone who “applies their knowledge of data science and machine learning to implement and run machine learning workloads...” Finally, AWS describes an **AWS Certified Machine Learning specialist** as someone with “the ability to design, implement, deploy, and maintain machine learning solutions for given business problems.”

One way to look at data science versus machine learning engineering is to consider science versus engineering itself. Science gears toward research, and engineering gears toward production. As machine learning moves beyond just the research side, companies are eager for a return on investment in hiring around AI and ML. According to payscale.com and glassdoor.com, the results show that at the end of 2020, the median salary for a data scientist, data engineer, and machine learning engineer was similar. According to LinkedIn in Q4 of 2020, 191K jobs mentioned cloud, there were 70K data engineering job listings, 55k machine learning engineering job listings, and 20k data science job listings, as shown in **Figure 1-1**.

Another way to look at these job trends is that they are a natural part of the hype cycle of technology. Organizations realize that to generate a return on investment (ROI), they need employees with hard skills: cloud computing, data engineering, and machine learning. They also need them in a much larger quantity than data scientists. As a result, the decade of the 2020s may show an acceleration in treating data science as a behavior, rather than a job title. DevOps is behavior, just like data science. Think about the principles of both DevOps and data science. In both cases, DevOps and data science are methodologies for evaluating the world, not necessarily unique job titles.

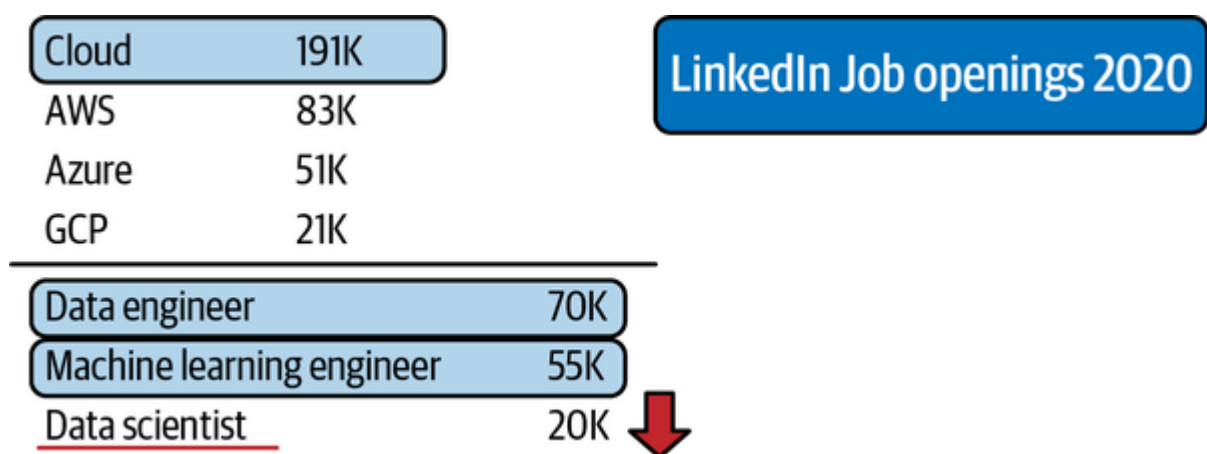


Figure 1-1. Machine learning jobs

Let’s look at the options for measuring the success of a machine learning engineering initiative at an organization. First, you could count the machine learning models that go into production. Second, you could measure the impact of the ML models on business ROI. These metrics culminate in a model’s operational efficiency. The cost, uptime, and staff needed to maintain it are signals that predict a machine learning engineering project’s success or failure.

Advanced technology organizations know they need to leverage methodologies and tools that decrease the risk of machine learning projects failing. So what are these tools and processes used in machine learning engineering? Here is a partial list:

Cloud native ML platforms

AWS SageMaker, Azure ML Studio, and GCP AI Platform

Containerized workflows

Docker format containers, Kubernetes, and private and public container registries

Serverless technology

AWS Lambda, AWS Athena, Google Cloud Functions, Azure Functions

Specialized hardware for machine learning

GPUs, Google TPU (TensorFlow Processing Unit), Apple A14, AWS Inferentia Elastic inference

Big data platforms and tools

Databricks, Hadoop/Spark, Snowflake, Amazon EMR (Elastic Map Reduce), Google Big Query

One clear pattern about machine learning is how deeply tied it is to cloud computing. This is because the raw ingredients of machine learning happen to require massive compute, extensive data, and specialized hardware. Thus, there is a natural synergy with deep integration with cloud platforms and machine learning engineering. Further supporting this is that the cloud platforms are building specialized platforms to enhance ML's operationalization. So, if you are doing ML engineering, you are probably doing it in the cloud. Next, let's discuss how DevOps plays a role in doing this.

What Is MLOps?

Why isn't machine learning 10X faster? Most of the problem-building machine learning systems involve everything surrounding machine learning modeling: data engineering, data processing, problem feasibility, and business alignment. One issue with this is a focus on the "code" and technical details versus solving the business problem with machine learning. There is also a lack of automation and the issue of HiPPO (Highest Paid Person's Opinions) culture. Finally, much of machine learning is not cloud native and uses academic datasets and academic software packages that don't scale for large-scale problems.

The quicker the feedback loop (see Kaizen), the more time to focus on business problems like the recent issues of rapid detection of Covid, detecting mask versus nonmask computer vision solutions deployed in the real world, and faster drug discovery. The technology exists to solve these problems, yet these solutions aren't available in the real world? Why is this?

NOTE

What is Kaizen? In Japanese, it means improvement. Using Kaizen as a software management philosophy originated in the Japanese automobile industry after World War II. It underlies many other techniques: Kanban, root cause analysis & five why's, and Six Sigma. To practice Kaizen, an accurate and realistic assessment of the world's state is necessary and pursues daily, incremental improvements in the pursuit of excellence.

The reason models are not moving into production is the impetus for the emergence of MLOps as a critical industry standard. MLOps shares a lineage with DevOps in that DevOps philosophically demands automation. A common expression is *if it is not automated, it's broken*. Similarly, with MLOps, there must not be components of the system that have humans as the levers of the machine. The history of automation shows that humans are the least valuable doing repetitive tasks but are the most valuable using technology as the architects and practitioners. Likewise, coordination between developers, models, and operations must coordinate through transparent teamwork and healthy collaboration. Think of MLOps as the process of automating machine learning using DevOps methodologies.

NOTE

What is DevOps? It combines best practices, including microservices, continuous integration, and continuous delivery, removing the barriers between Operations and Development and Teamwork. You can read more about DevOps in our book *Python for DevOps* (O'Reilly). Python is the predominant language of scripting, DevOps, and machine learning. As a result of this, this MLOps book focuses on Python, just as the DevOps book focused on Python.

With MLOps, not only do the software engineering processes need full automation, but so do the data and modeling. The model training and deployment is a new wrinkle added to the traditional DevOps lifecycle. Finally, additional monitoring and instrumentation must account for new things that can break, like data drift—the delta between changes in the data from the last time the model training occurred.

A fundamental problem in getting machine learning models into production is the immaturity of the data science industry. The software industry has embraced DevOps to solve similar issues; now, the machine learning community embraces MLOps. Let's dive into how to do this.

DevOps and MLOps

DevOps is a set of technical and management practices that aim to increase an organization's velocity in releasing high-quality software. Some of the benefits of DevOps include speed, reliability, scale, and security. These benefits occur through adherence to the following best practices:

Continuous integration (CI)

CI is the process of continuously testing a software project and improving the quality based on these tests' results. It is automated testing using open source and SaaS build servers such as GitHub Actions, Jenkins, Gitlab, CircleCI, or cloud native build systems like AWS Code Build.

Continuous delivery (CD)

This method delivers code to a new environment without human intervention. CD is the process of deploying code automatically, often through the use of IaC.

Microservices

A microservice is a software service with a distinct function that had little to no dependencies. One of the most popular Python-based microservice frameworks is Flask. For example, a machine learning prediction endpoint is an excellent fit for a microservice. These microservices can use a wide variety of technologies, including FaaS (function as a service). A perfect example of a cloud function is AWS Lambda. A microservice could be container-ready and use CaaS (container as a service) to deploy a Flask application with a Dockerfile to a service like AWS Fargate, Google Cloud Run, or Azure App Services.

Infrastructure as Code

Infrastructure as Code (IaC) is the process of checking the infrastructure into a source code repository and “deploying” it to push changes to that repository. IaC allows for idempotent behavior and ensures the infrastructure doesn't require humans to build it out. A cloud environment defined purely in code and checked into a source control repository is a good example use case. Popular technologies include cloud-specific IaC like AWS Cloud Formation or **AWS SAM (Serverless Application Model)**. Multicloud options include **Pulumi** and **Terraform**.

Monitoring and instrumentation

Monitoring and instrumentation are the processes and techniques used that allow an organization to make decisions about a software system's performance and reliability. Through logging and other tools like application performance monitoring tools such as New Relic, Data Dog, or Stackdriver, monitoring and instrumentation are essentially collecting data about the behavior of an application in production or data science for deployed software systems. This process is where Kaizen comes into play; the data-driven organization uses this instrumentation to make things better daily or weekly.

Effective technical communication

This skill involves the ability to create effective, repeatable, and efficient communication methods. An excellent example of effective technical communication could be adopting AutoML for the

initial prototyping of a system. Of course, ultimately, the AutoML model may be kept or discarded. Nevertheless, automation can serve as an informational tool to prevent work on an intractable problem.

Effective technical project management

This process can efficiently use human and technology solutions, like ticket systems and spreadsheets, to manage projects. Also, appropriate technical project management requires breaking down problems into small, discrete chunks of work, so incremental progress occurs. An antipattern in machine learning is often when a team works on one production machine model that solves a problem “perfectly.” Instead, smaller wins delivered daily or weekly is a more scalable and prudent approach to model building.

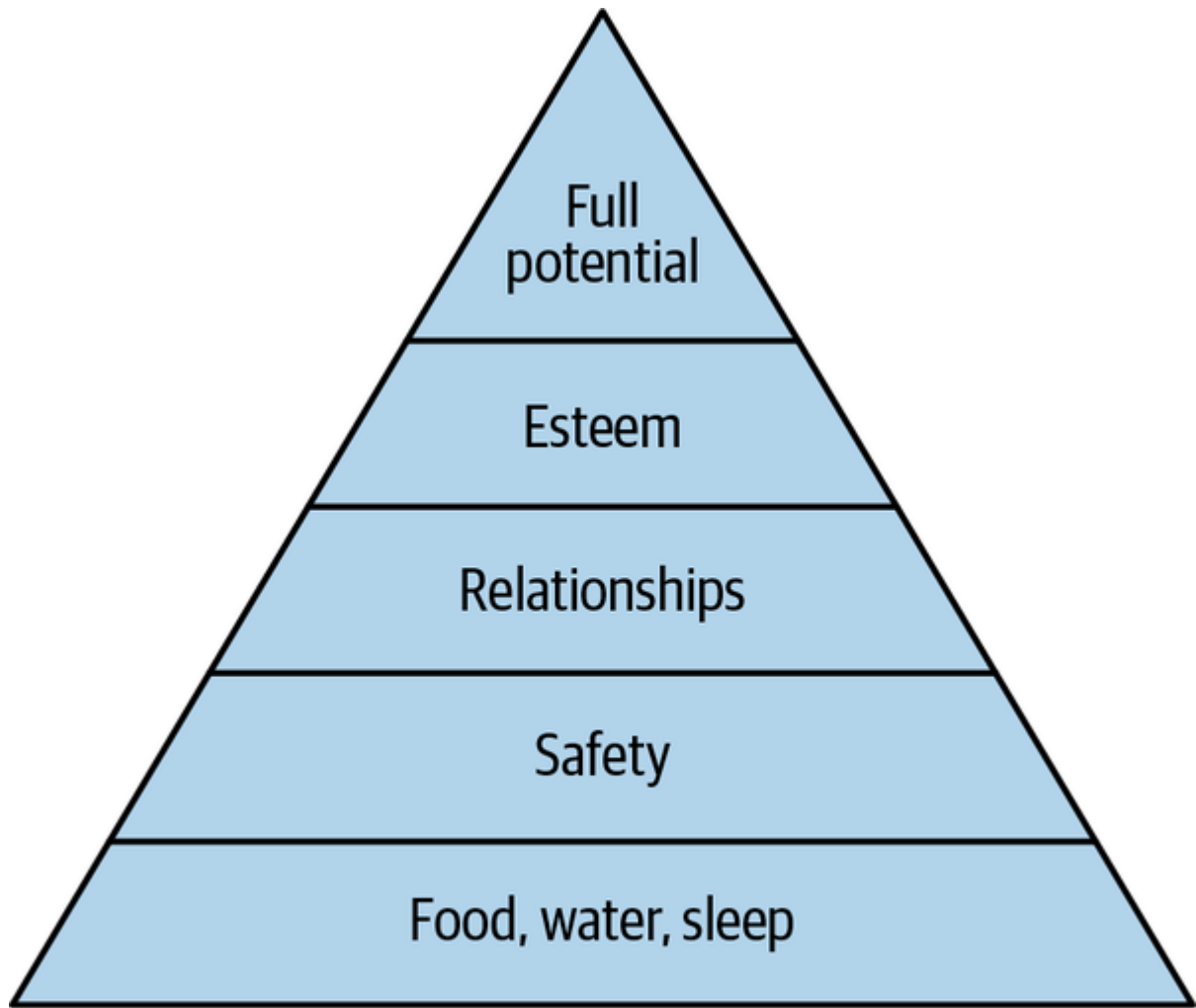
Continuous integration and continuous delivery are two of the most critical pillars of DevOps. Continuous integration involves merging code into a source control repository that automatically checks the code’s quality through testing. Continuous delivery is when code changes are automatically tested and deployed, either to a staging environment or production. Both of these techniques are a form of automation in the spirit of Kaizen or continuous improvement.

A good question to ask is who on the team should implement CI/CD? This question would be similar to asking who contributes to taxes in a democracy. In a democracy, taxes pay for roads, bridges, law enforcement, emergency services, schools, and other infrastructure, so all must contribute to building a better society. Likewise, all MLOps team members should help develop and maintain the CI/CD system. A well-maintained CI/CD system is a form of investment in the future of the team and company.

An ML system is also a software system, but it contains a unique component: a machine learning model. The same benefits of DevOps can and do apply to ML systems. The embrace of automation is why new approaches like Data Versioning and AutoML hold many promises in capturing the DevOps mindset.

An MLOps Hierarchy of Needs

One way to think about machine learning systems is to consider Maslow’s hierarchy of needs, as shown in **Figure 1-2**. Lower levels of a pyramid reflect “survival,” and true human potential occurs after basic survival and emotional needs are met.



Maslow's hierarchy of needs

Figure 1-2. Maslow's hierarchy of needs theory

The same concept applies to machine learning. An ML system is a software system, and software systems work efficiently and reliably when DevOps and data engineering best practices are in place. So how could it be possible to deliver the true potential of machine learning to an organization if DevOps' basic foundational rules don't exist or data engineering is not fully automated? The ML hierarchy of needs shown in [Figure 1-3](#) is not a definitive guide but is an excellent place to start a discussion.

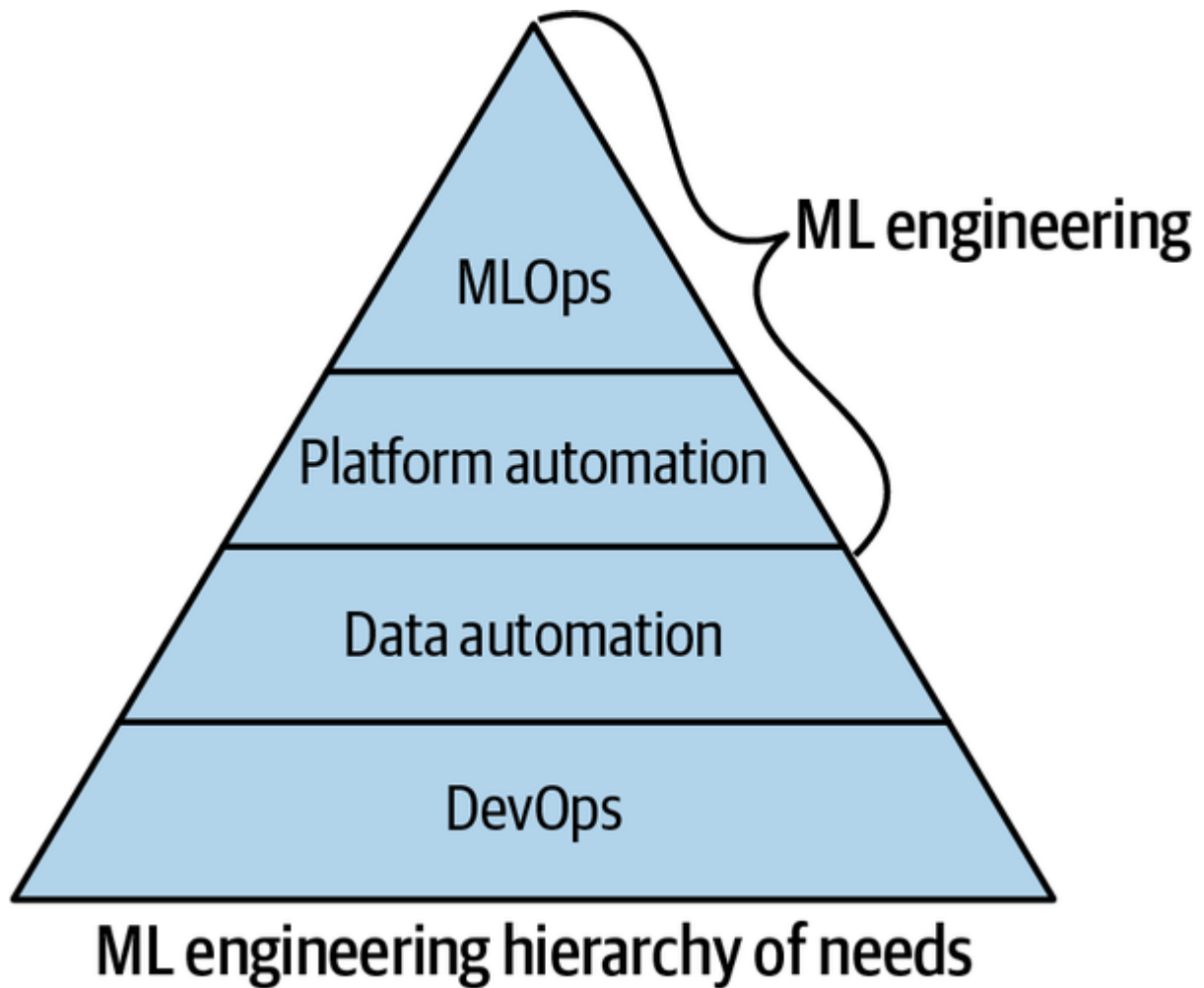


Figure 1-3. ML engineering hierarchy of needs

One of the major things holding back machine learning projects is this necessary foundation of DevOps. After this foundation is complete, next is data automation, then platform automation, and then finally true ML automation, or MLOps, occurs. The culmination of MLOps is a machine learning system that works. The people that work on operationalizing and building machine learning applications are machine learning engineers and/or data engineers. Let's dive into each step of the ML hierarchy and make sure you have a firm grasp of implementing them, starting with DevOps.

This document was truncated here because it was created in the Evaluation Mode.